Decision Rules for Streaming LLM Layers under Memory Pressure: Dense vs Mixture-of-Experts

Abderrahmen Bejaoui · 7 October 2026 · repository: seneca-innovation-center/k8s-llm-offload-bench

Test machine: AMD Ryzen AI Max+ 395 (16 cores), 125 GB unified RAM, one NVMe SSD on a PCIe 4.0 ×4 link (7.1 GB/s measured). Single-node Kubernetes (k3d) pinned to 16 hardware threads. PyTorch on CPU, bf16 weights, no quantisation.

Data: 1,431 measured runs on six models: Qwen2.5-1.5B, 3B, 7B and 14B (dense), gpt-oss-20b and Qwen3-30B-A3B (mixture of experts).

Claim. Across six models, the best way to serve a model larger than the free memory depends on its architecture. For dense models, streaming the layers from one pod with tree speculation gives the best latency for the memory saved. For mixture-of-experts models, kernel paging under a pod memory limit gives the lowest latency for savings up to about 70%. Spreading layers over several pods never beats the disk-bandwidth floor, which four concurrent reads already reach.

---

1. Executive decision table

Costs are per generated token for one user, relative to the same model fully resident in one pod. Ranges cover three prompts (a 512-token prose text and two chat prompts). Evidence IDs are sections of `DETAILED_REPORT.md`. Every number below is printed by `python3 scripts/reproduce_decision_table.py`.

Situation	Recommended approach	Cost per token vs resident	Memory saved	Evidence
Dense model, one user, RAM tight	Stream layers from one pod (4 read slots) + 64-node tree speculation with a small drafter	1.5–2.9× (3B–14B); 1.7–2.1× at 1.5B	51–74% (3B–14B); 24–28% at 1.5B	E1, D6
Dense model, many users	Streaming + batching	1.19× per user at 512 users; 84% of resident throughput (3B, 4-pod streaming)	32 of 36 layers off memory	B9
Mixture of experts, one user	Kernel paging: memory-mapped weights + pod memory limit	Qwen3-30B-A3B: 1.7× at 38–39%, 2.6–3.3× at 57–58%, 4.3–5.2× at 72–73% saved. gpt-oss-20b: 3.0–4.0× at 29–30%, 7.4–8.0× at 54% saved	29–73%	E1, E4, E5
Mixture of experts, many users	Not measured with paging. Layer streaming + batching measured	1.78× per user at 512 users; 56% of throughput (gpt-oss, 4 pods)	20 of 24 layers off memory	C8
Dense model, kernel paging	**Do not use**	8.5–14.9×		E1, B7
Mixture of experts, layer streaming	**Do not use**	31.8–37.2×		E1, C3
Partial residency (pin most layers, stream a few)	Only for savings under ~10%	1.33× (3B) and 3.9× (gpt-oss) before saving anything	0%	B6, C6
Several pods vs one pod with read slots	**One pod**	Same floor: 781 ms (4 pods) vs 798 ms (1 pod, 4 slots), 3B	3.25 GB vs 2.14 GB total RAM	B10, C5

> **Rule D (dense).** Stream the layers from one pod with four read slots and add tree speculation with a small drafter from the same model family. The drafter's memory comes off the saving, so it must be small next to the target. Never use kernel paging on a dense model.

> **Rule M (mixture of experts).** Memory-map the weights and give the pod a memory limit. The kernel keeps the experts that tokens actually use. Never stream whole layers: a token uses 6–12% of a layer's experts, but streaming reads all of them.

> **Rule F (floor).** Any scheme that re-reads all layer weights per pass is bounded by layer bytes ÷ disk bandwidth per pass. Four concurrent reads reach the bandwidth. More pods or readers add memory, not speed.

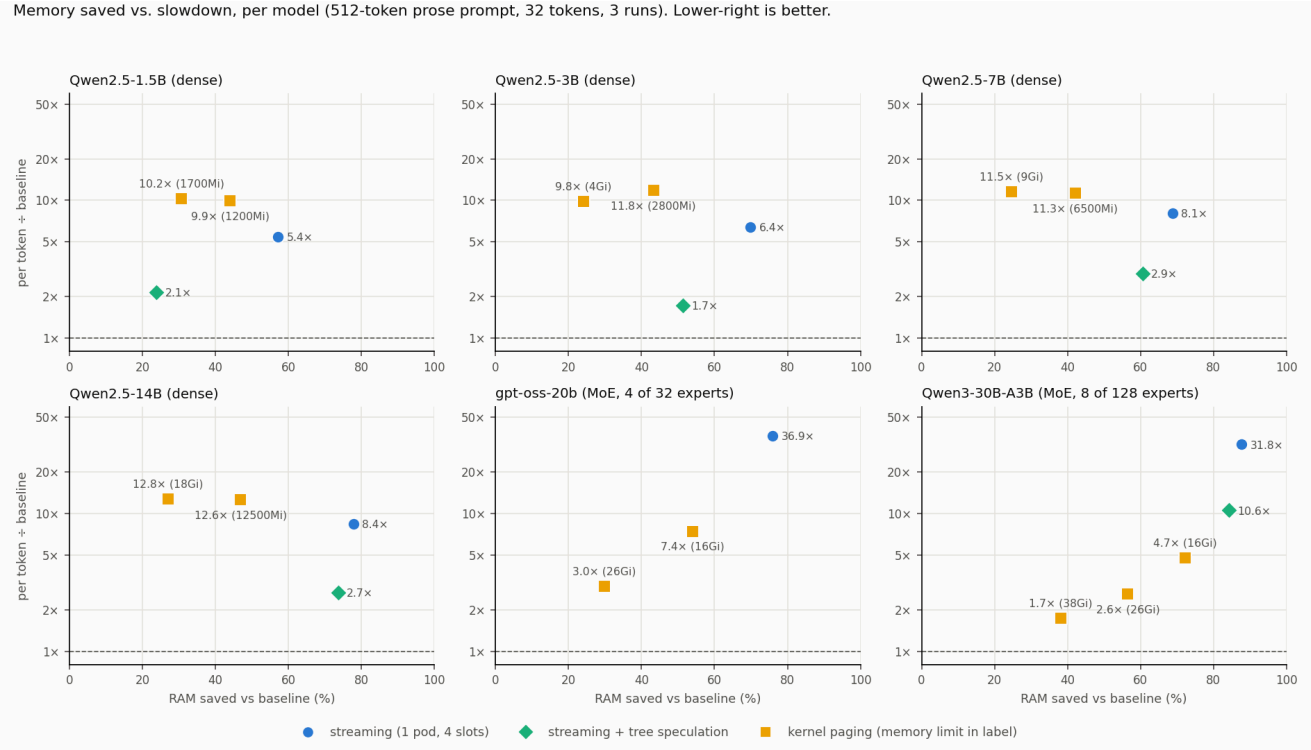


Figure 1. Memory saved against slowdown for every configuration of every model (512-token prose prompt, 32 tokens, 3 runs). Lower-right is better. Dense models: the tree-speculation diamond is lowest. Mixtures of experts: the kernel-paging squares are.

2. Problem and scope

A node has limited RAM and one fast NVMe drive. The operator must serve a model whose weights do not fit in the RAM that is free, without quantising it. Which placement strategy gives the best latency for the memory saved?

Five techniques were compared on the same node, with the same code and the same measurement method:

1. **Cyclic multi-pod streaming.** Layers are spread over K pods. Each pod computes a layer, frees it, and loads its next one from disk while the other pods compute.
2. **Single-pod streaming.** One pod computes the layers in order and prefetches the next layers through S read slots.
3. **Partial residency.** Most layers stay in memory; the last few are streamed.

4. **Kernel paging.** Weights are memory-mapped and the pod gets a Kubernetes memory limit. Linux keeps what fits and re-reads the rest on demand.
5. **Speculative decoding on top of streaming.** A small drafter proposes a chain, and later a tree, of candidate tokens. One streamed pass of the big model verifies them all and keeps exactly the tokens it would have produced itself.

Batching was measured as a sixth axis for many concurrent users.

Out of scope: quantisation, multi-node serving, discrete GPUs, continuous-batching schedulers, and generations longer than 48 tokens.

3. Measurement principles

- **Baseline:** the whole model resident in one pod, measured in the same round as the configuration it is compared with. The machine was shared with other workloads, so only comparisons within a round are used.
- **Per token:** median decode time per generated token for one user. For speculative runs, total decode time ÷ tokens produced, the honest number when one read yields several tokens.
- **First token:** time from receiving the prompt to the first generated token (prefill).
- **RAM:** peak resident memory (high-water mark) of all pods, coordinator included, in decimal GB.
- **Same output:** greedy decoding in bf16. Every run's tokens are compared with the baseline's. Divergences are checked against a full-precision (fp32) reference.
- **Cold reads:** streamed weight files are evicted from the page cache around every read, so a reload is a real disk read. For kernel paging the cache is dropped after deploying and before measuring.
- **Protocol:** 3 repetitions per configuration. 16–48 generated tokens depending on the experiment; 32 in the six-model test.

4. The hardware floor

Two microbenchmarks fix the constants that every later result depends on.

One layer	Qwen2.5-3B (154 MB)	gpt-oss-20b (1.65 GB)
Load from disk, one reader	41 ms	598 ms
Compute, 1 token	3.5 ms	6.4 ms
Compute, 2048 tokens	166 ms	388 ms
Load ÷ compute for one token	12×	94×

Parallel readers	1	2	4	8	12	18
Total disk bandwidth (GB/s)	4.14	6.09	7.01	7.11	7.12	7.11

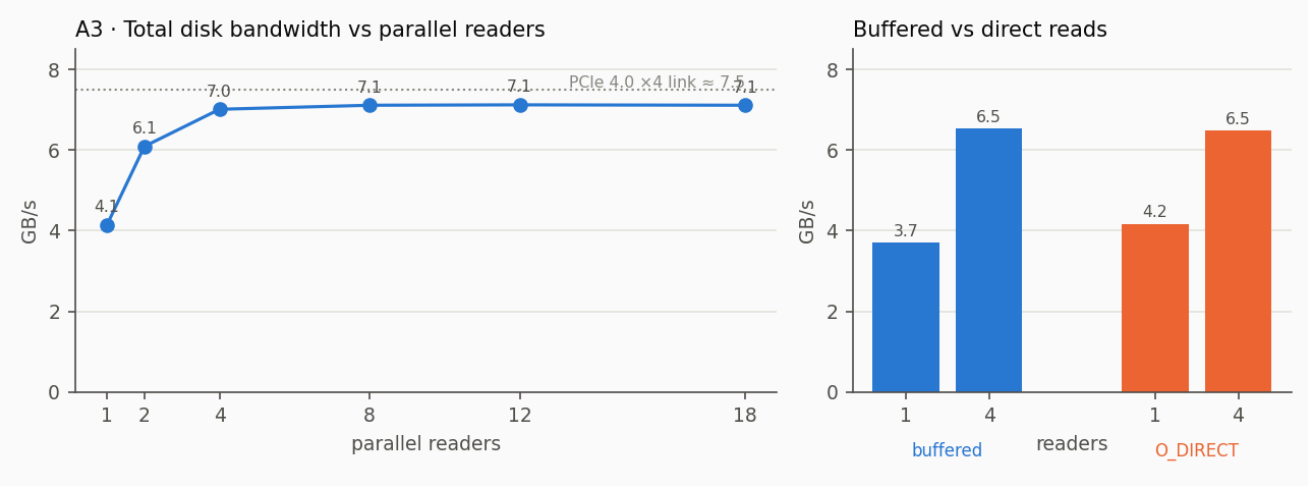


Figure 2. Total disk bandwidth against parallel readers, and buffered against direct reads (A3).

Generation computes one token per pass, so a layer computes 12 to 94 times faster than it loads. The drive is capped by its PCIe 4.0 ×4 link: four readers reach 7.0 GB/s and nothing goes faster. Direct I/O was measured too and does not raise the bandwidth (A3, D2).

The consequence is a floor. A scheme that re-reads all layer weights for each generated token cannot beat **layer bytes ÷ 7.1 GB/s** per token. Plain single-pod streaming sits on that floor for all six models:

Model	Layer bytes	Predicted floor	Measured streaming	Measured ÷ predicted
Qwen2.5-1.5B	2.62 GB	369 ms	370–378 ms	1.00–1.03
Qwen2.5-3B	5.55 GB	782 ms	790–792 ms	1.01
Qwen2.5-7B	13.05 GB	1838 ms	1866–1882 ms	1.01–1.02
Qwen2.5-14B	26.43 GB	3722 ms	3815–3907 ms	1.03–1.05
gpt-oss-20b	39.51 GB	5565 ms	5550–5571 ms	1.00
Qwen3-30B-A3B	59.82 GB	8426 ms	8368–8480 ms	0.99–1.01

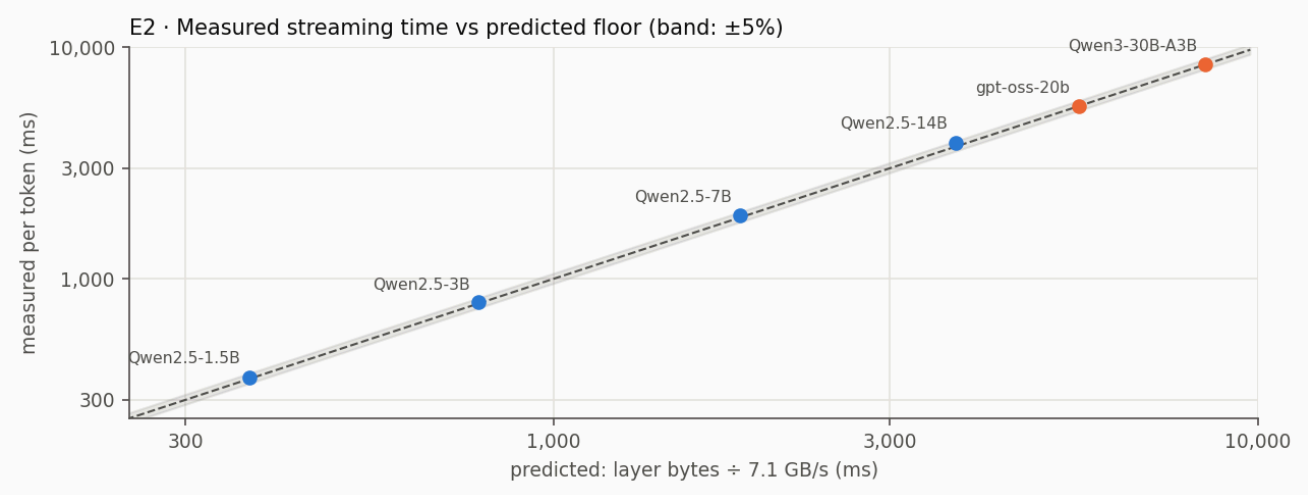


Figure 3. Measured streaming time against the predicted floor, six models, band ±5% (E2).

The rule is predictive. For any model on a given node, the bytes that must be read per token divided by the drive's saturated bandwidth give the minimum time per token. Every technique below either accepts this floor, keeps bytes in memory, or gets more than one token out of each read.

5. Decision Rule D: dense models

5.1 Streaming saves memory but sits on the floor

The original idea, cyclic streaming over pods, delivers its memory promise on Qwen2.5-3B: with 2 pods only 2 of 36 layers are in memory at any moment, 68% less total RAM. Generation does not keep its speed:

Qwen2.5-3B, 512-token prompt	Total RAM	Per token	vs resident
Resident baseline	7.07 GB	133 ms	1.0×
Cyclic streaming, 2 pods	2.26 GB	1013 ms	7.6×
Cyclic streaming, 4 pods	3.25 GB	781 ms	5.9×
Single pod, 4 read slots	2.14 GB	798 ms	6.0×

Long prompts hide the loads: with a 2048-token prompt the first token takes 1.00× the baseline, because each layer computes for 166 ms while a load takes 41 ms (B2). Generation is where streaming pays.

5.2 Why more pods do not help

The intuition was that with enough pods the next layer is always loaded when the token comes back; the microbenchmark (A2) suggests about 13 pods. The sweep went to 18 pods and the time stayed flat at the floor (B3, B4):

- **All pods share one drive.** Each pod starts its next load as soon as it finishes computing, so about K reads are in flight. Each load slows down as pods are added, from 49 ms with 2 pods to 318 ms with 18. A pod adds as much load time as it adds cover.
- **The drive saturates at four readers.** From 4 pods on the time is flat at 0.78 s per token.
- **Loading slows computing.** Disk transfers and matrix multiplications share memory bandwidth: one-token compute slows 1.44× while layers stream in (D2).

Grouping several layers per pod changes nothing: every 2-pod configuration is about 1.0 s per token, every configuration with 4 or more pods 0.77–0.79 s (B4).

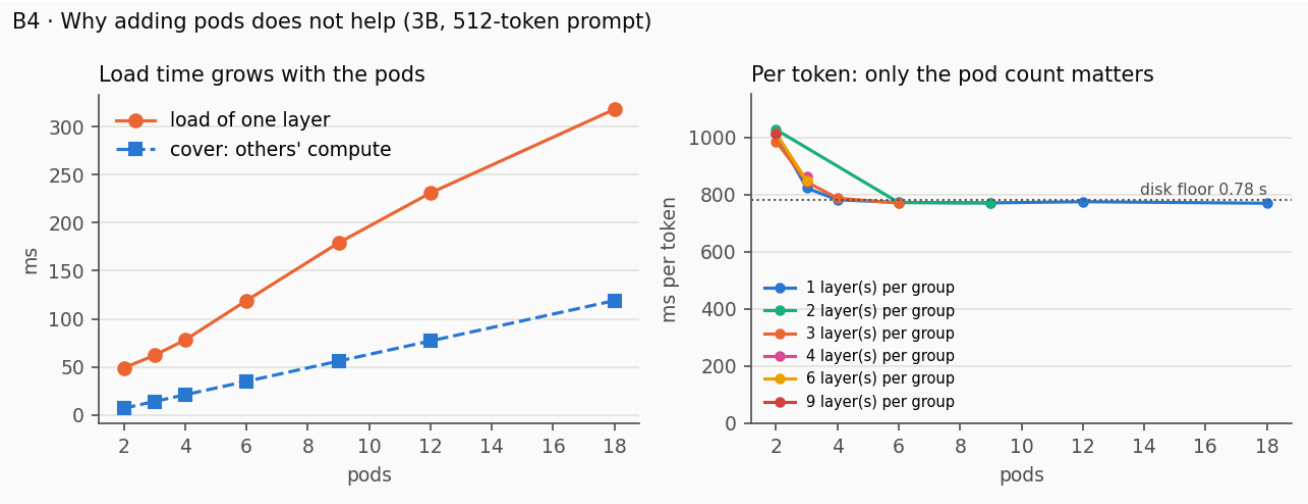


Figure 4. Left: one layer's load time grows with the pod count faster than the cover from the other pods. Right: per token, only the pod count matters, and it bottoms out at the disk floor (B4).

5.3 Speculation is the lever

The verify pass of speculative decoding is disk-bound: it costs about the same as a plain token whether it checks 1 candidate or 65. So the only way below the floor per token is more tokens per read. Adding a 1 GB drafter to cyclic streaming and then optimising for four rounds cut the time per token at about half the resident memory (D1–D6):

Step (Qwen2.5-3B)	Total RAM	Prose 512	Chat: explain	Chat: summary	Chat: code
Cyclic, 2 pods, no speculation	2.26 GB	1013 ms (7.6×)	n/a	n/a	n/a
+ chain of 5 drafted tokens, 2 pods	3.48 GB	441 ms (3.3×)	n/a	n/a	n/a
Chain, one pod with 4 slots	3.1–3.4 GB	350 ms (2.7×)	315 ms (2.9×)	301 ms (2.4×)	260 ms (2.3×)
Tree of 23–65 candidates, Hugging Face drafter	3.2–3.5 GB	324 ms (2.6×)	237 ms (2.15×)	245 ms (2.0×)	217 ms (1.95×)
65-node tree, lean drafter in the same layer code	3.2–3.4 GB	**242 ms (1.9×)**	**248 ms (2.2×)**	**200 ms (1.7×)**	**198 ms (1.7×)**
Resident baselines	7.0–7.1 GB	126–133 ms	110–115 ms	118–123 ms	111–115 ms

Ratios use the baseline of the round each step was measured in. The tree yields 4.4–5.5 tokens per disk read where the chain gave 2.4–3.8. Larger trees (129 and 201 nodes) and direct I/O were measured and lost (D3, D5).

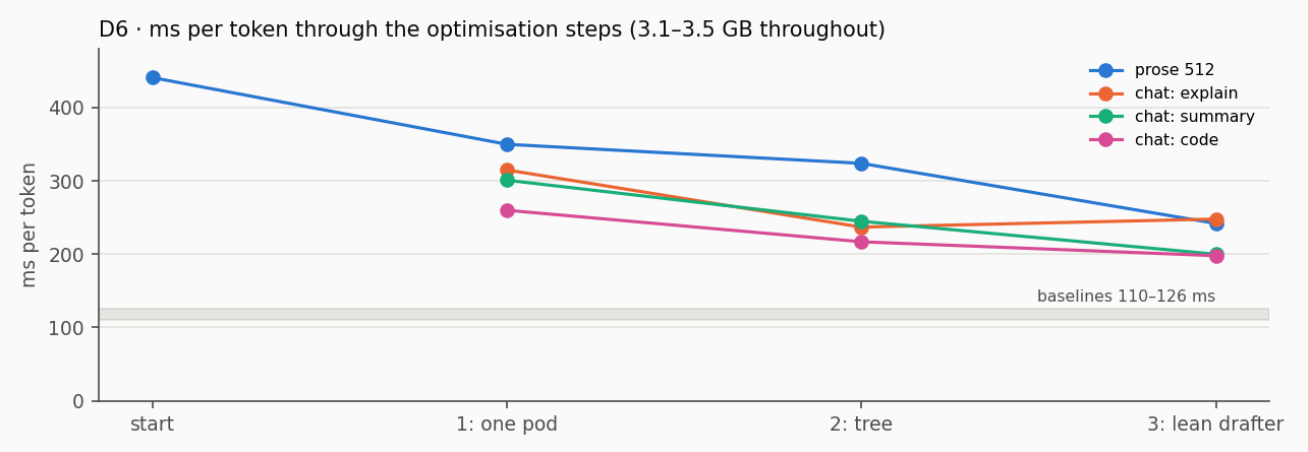


Figure 5. Time per token through the optimisation steps, at 3.1–3.5 GB (D6).

5.4 The rule holds on four dense models

The same protocol on four dense models (three prompts each, 32 tokens, 3 repetitions):

Model	Streaming, 4 slots	Streaming + tree speculation	Kernel paging (each limit)
Qwen2.5-1.5B	5.4–5.9× at 57–59% saved	1.7–2.1× at 24–28% saved	10.2–11.1× at 29–31%; 9.9–10.9× at 43–44%
Qwen2.5-3B	6.4–7.0× at 70–71% saved	1.5–1.9× at 51–54% saved	8.5–12.3× at 23–24%; 11.8–12.6× at 42–43%
Qwen2.5-7B	8.1–8.6× at 69% saved	1.7–2.9× at 61–62% saved	9.4–11.9× at 24–25%; 11.2–11.9× at 42%
Qwen2.5-14B	8.4–9.6× at 78% saved	2.1–2.7× at 74% saved	12.2–14.9× at 26–27%; 12.6–14.1× at 46–47%

Tree speculation wins on every dense model and every prompt. Kernel paging is the worst option every time: a dense token touches every weight, so once the model does not fit, every token re-faults the whole model in small pieces. The drafter is the constraint at small sizes. The 1 GB Qwen2.5-0.5B drafter is a third of the 1.5B model and leaves only 24–28% saving; from 7B up it costs 7% of the model or less.

> **Rule D.** For a dense model under memory pressure, stream the layers from one pod with four read slots and add tree speculation with a drafter much smaller than the target. Expect 1.5–2.9× the resident time per token at 51–74% less memory (3B–14B). Do not use kernel paging.

6. Decision Rule M: mixture of experts

6.1 Memory still drops, layer streaming collapses

On gpt-oss-20b (32 experts per layer, 4 used per token), cyclic streaming saves memory even better than on the 3B: 2 pods use 90% less stage-pod RAM, because the per-pod runtime is small next to 1.65 GB layers (C1). Generation, however, is 33–42× slower and flat at the 5.55 s floor from 4 pods on (C3).

The mixture of experts adds a cost on top of the dense case. Which 4 experts a token needs is decided by the router only when the token reaches the layer. A prefetching scheme must therefore load all 32, reading 8× more bytes than the token uses. Even a 2048-token prompt no longer hides the loads: the first token takes 1.44× the baseline, because streaming 1.65 GB layers slows each layer's compute by 30–55% (C2).

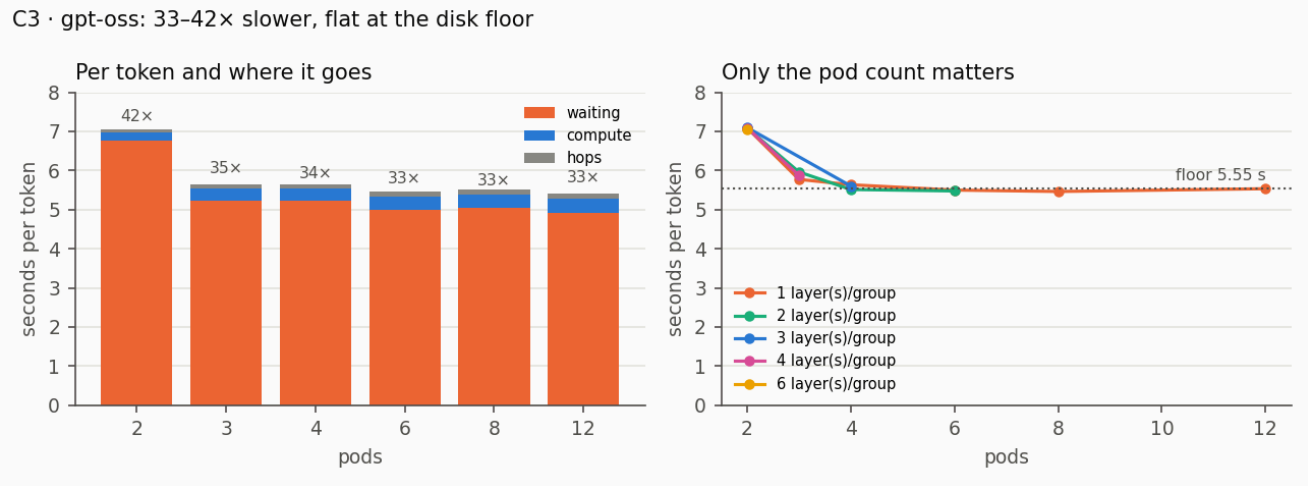


Figure 6. gpt-oss-20b: time per token and its breakdown by pod count, and the same for every layer grouping. Flat at the 5.55 s floor (C3).

6.2 Kernel paging is the winner

Kernel paging reads only the experts that tokens touch, and the kernel keeps the recently used ones within the pod's memory limit:

Model	Streaming, 4 slots	Streaming + tree speculation	Kernel paging (each limit)
gpt-oss-20b	35.4–37.2× at 76–77% saved	no drafter in the model family	26 GiB: 3.0–4.0× at 29–30%; 16 GiB: 7.4–8.0× at 54%
Qwen3-30B-A3B	31.8–34.9× at 88% saved	6.7–10.6× at 84–85% saved	38 GiB: 1.7× at 38–39%; 26 GiB: 2.6–3.3× at 57–58%; 16 GiB: 4.3–5.2× at 72–73%

Qwen3-30B-A3B uses 8 of 128 experts per token (6%), gpt-oss-20b 4 of 32 (12%), and paging is cheaper on the sparser model. Tree speculation does not rescue streaming here: it yields 3–5 tokens per read, but each read is 60 GB. It only becomes the choice when more than about 75% of the memory must be saved, at 6.7–10.6× the resident time.

The price of paging is the first token. The first pass touches nearly every expert, so the first token takes 3.5–6.2× the baseline on the 512-token prompt and up to 11× on short prompts (E3). A service using paging on a mixture of experts should warm up each pod before taking traffic.

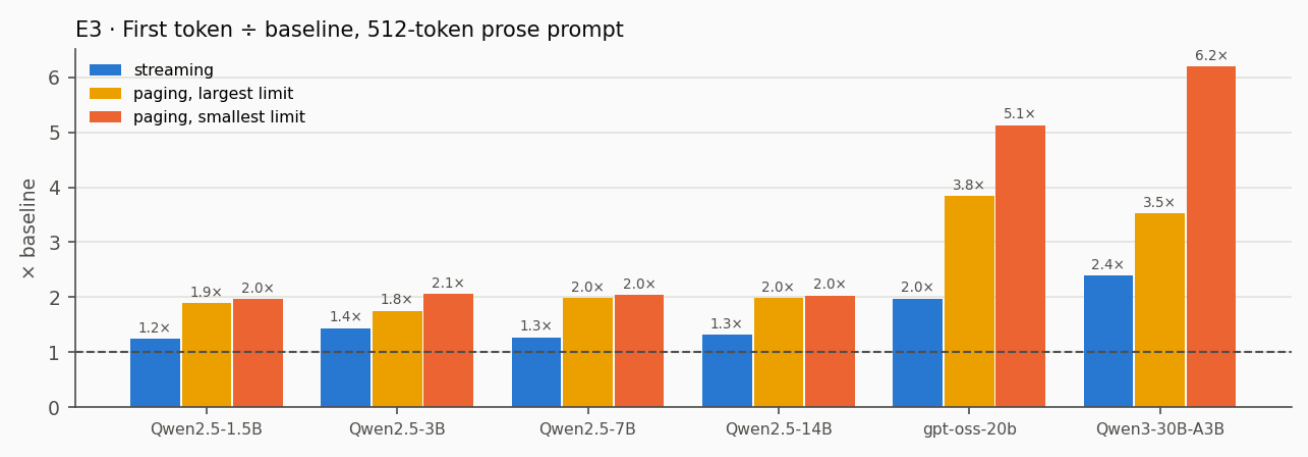


Figure 7. First token against the resident baseline, 512-token prose prompt: streaming, and kernel paging at the largest and smallest limit (E3).

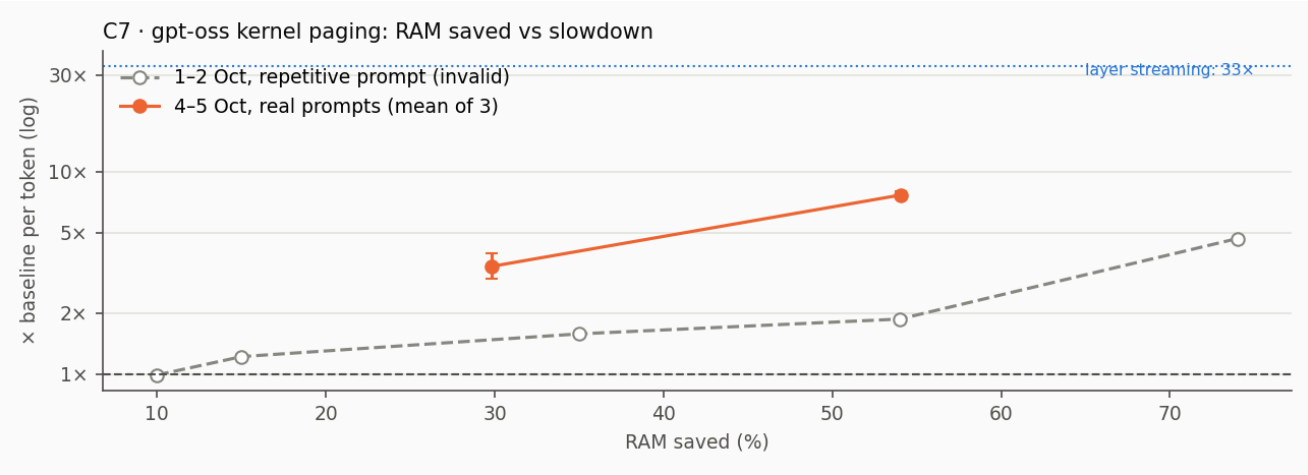


Figure 8. gpt-oss-20b kernel paging: memory saved against slowdown. Grey: the first measurement, invalid (§6.3). Orange: real prompts, mean and range over three prompts (C7).

6.3 A measurement trap: degenerate output

The first gpt-oss paging measurement reported 54% less memory at 1.87×. That figure was wrong. Those runs used a repetitive prompt on which gpt-oss generates the token "…" sixteen times in a row. A repeated token routes to the same experts at every step, so the cache hit almost every time. With real prompts the same limit costs 7.4–8.0×.

Two controlled reruns settled it. Re-running with the old prompt reproduced 1.85×. Clearing the host page cache (79 GB of unrelated cached files) and re-measuring all 15 paging configurations of both mixtures of experts reproduced the sweep within 10%, so host cache pressure was not the cause (E4, E5). The lesson transfers to any cache-sensitive benchmark: check that the generated tokens are not degenerate before trusting the number.

6.4 Dense vs mixture of experts, side by side

	Qwen2.5-3B (dense)	gpt-oss-20b (experts)
One layer: load ÷ compute (1 token)	12×	94×
Bytes loaded per token ÷ bytes used	1	8
Disk floor per token	0.78 s	5.55 s
Cyclic streaming, 4+ pods	6×	33×
First token, 2048-token prompt, best streaming	1.00×	1.44×
Compute slowdown while loading (prefill)	+8%	+30–55%

	Qwen2.5-3B (dense)	gpt-oss-20b (experts)
Partial residency, nothing saved	1.33×	3.9×
Kernel paging at ~54% less RAM	10.6×	7.4–8.0×

Every streaming approach gets worse in proportion to the bytes read per token. Only kernel paging improves relative to the others, because expert weights are touched sparsely.

> **Rule M.** For a mixture-of-experts model under memory pressure, memory-map the weights and set a pod memory limit; warm the pod up before serving. Expect 1.7× (Qwen3-30B-A3B, 38–39% saved) to 7.4–8.0× (gpt-oss-20b, 54% saved), depending on how sparse the routing is and how much memory is saved. Do not stream whole layers.

7. Supporting evidence

7.1 Correctness and output identity

Every layer implementation (Qwen2, gpt-oss, Qwen3, Qwen3-MoE) matches Hugging Face `transformers` to fp32 precision (A1). `scripts/validate_output_identity.py` compares every run's tokens with its baseline:

- **1,136 of 1,200** non-baseline runs produce exactly the baseline's tokens.
- Every streaming, partial-residency, kernel-paging and control run is identical.
- The 64 others are all speculative or batched runs. They diverge at one of eight specific tokens, never elsewhere.

At each of those eight tokens the full-precision model's top two candidates differ by only 0.05–0.27 logit points (0.20–1.16%). Computing the same position in a different shape moves bf16 logits by up to 0.28–0.42 points (median 0.16–0.23) on two reference models, more than every gap. Speculative and batched passes have a different shape from single-token decoding, so these are bf16 rounding ties, not bugs. The fp32 reference sides with the speculative run four times and with the baseline four times (F1–F3).

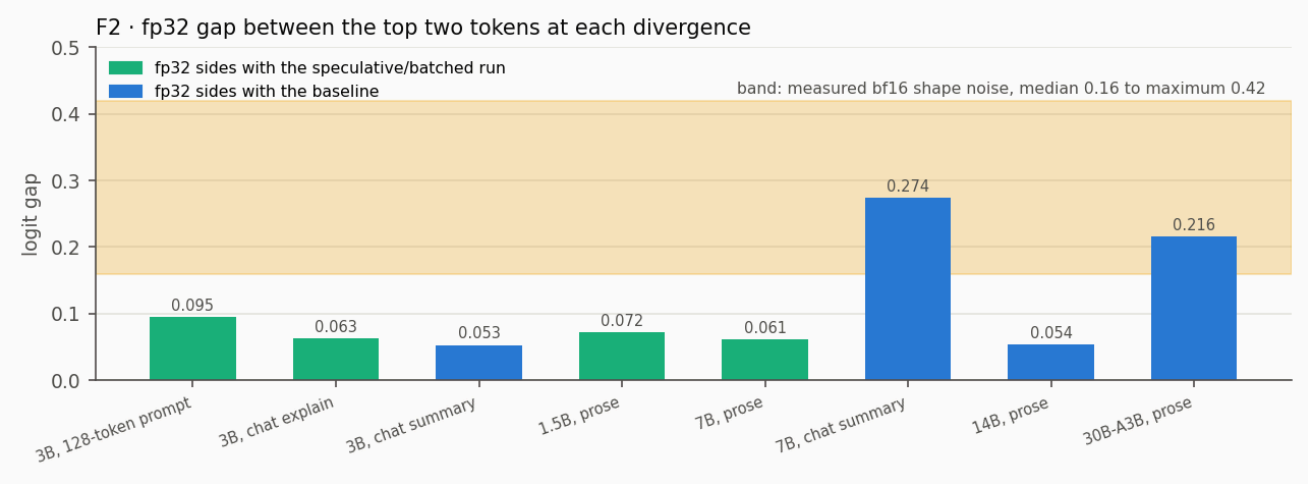


Figure 9. Full-precision gap between the top two tokens at each divergence, against the band of measured bf16 shape noise (F2).

7.2 One pod with read slots beats several pods

The pods were only ever a way to have several reads in flight. One pod with S slots follows exactly the same curve as S pods and uses less memory, because it runs one runtime and moves no activations between pods (B10, C5):

Configuration	Qwen2.5-3B: RAM, per token	gpt-oss-20b: RAM, per token
4 pods (cyclic)	3.25 GB, 781 ms	12.1 GB, 5.64 s
1 pod, 4 slots	2.14 GB, 798 ms	10.5 GB, 5.56 s
6 pods (cyclic)	4.26 GB, 772 ms	n/a
1 pod, 6 slots	2.44 GB, 778 ms	12.6 GB, 5.53 s

On Kubernetes this means one StatefulSet replica per model with an in-process prefetcher, not a pipeline of pods.

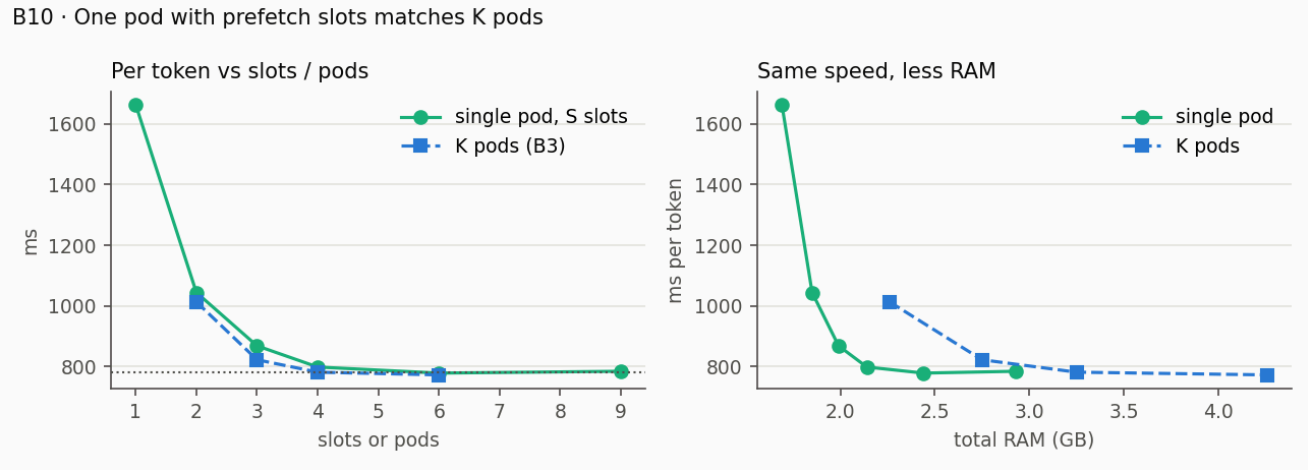


Figure 10. Qwen2.5-3B: one pod with S read slots against K pods, per token and against total RAM (B10).

7.3 Batching

One disk pass advances every request in a batch by one token, so the fixed read cost is shared (B9, C8):

Concurrent users	1	16	64	256	512
3B, 4-pod streaming: per user vs resident	6.4×	4.2×	1.9×	1.3×	1.19×
3B: share of resident throughput	16%	24%	53%	76%	84%
gpt-oss, 4-pod streaming: per user vs resident	34×	9.7×	5.7×	2.5×	1.78×
gpt-oss: share of resident throughput	3%	11%	17%	40%	56%

Batching closes the gap for a busy service but never brings a single request below one full read per token. It does not change the single-user ranking.

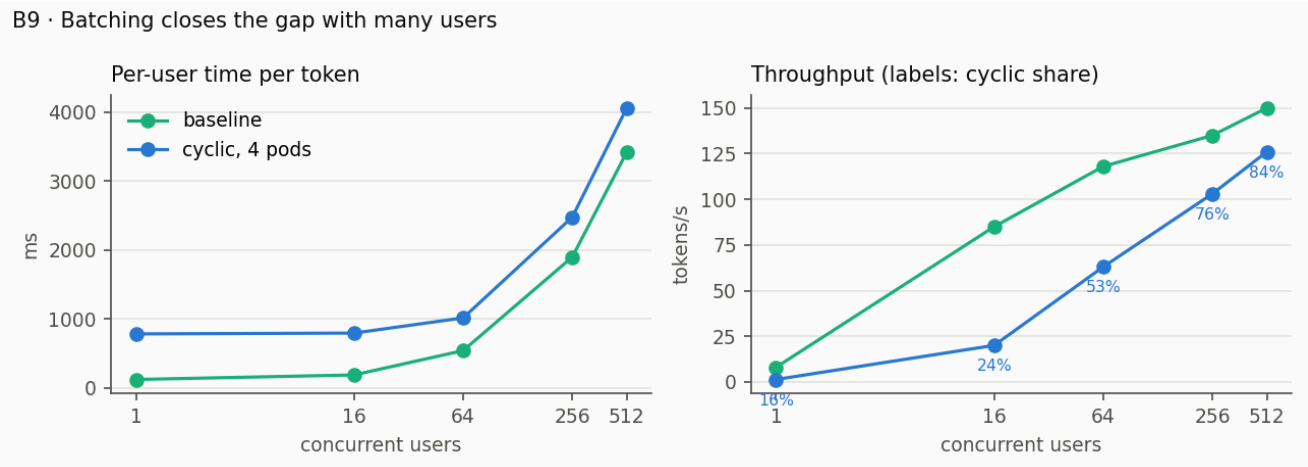


Figure 11. Batching on the 3B: per-user time and throughput against concurrent users (B9).

8. Limitations and transferability

- **One machine class.** Unified memory, one NVMe on PCIe 4.0 ×4, 16 CPU cores, no GPU used. The floor formula (bytes ÷ bandwidth) transfers to any storage. The constants do not: a node with several NVMe drives in RAID-0 or PCIe 5.0 has a higher bandwidth and a lower floor. Whether the ranking holds on a discrete-GPU node streaming from host memory is a hypothesis, not tested here.
- **Short generations.** 16–48 generated tokens per run and three prompts per model in the six-model test. Speculative acceptance and paging hit rates on long generations were not measured; paging warms up over a generation, so its long-run ratios are likely somewhat better than these medians.
- **Coarse paging limits.** Two or three memory limits per model. The dense-versus-experts contrast is a factor of 3–10, so the rule does not depend on the exact limits, but the optimal limit for a given model is not pinned down.
- **Combinations not measured.** Tree speculation with batching, and kernel paging with batching.
- **Shared machine.** Other workloads ran during some rounds. Baselines come from the same round as each comparison, and absolute numbers across rounds are not comparable.
- **bf16 only, no quantisation.** Quantised weights would cut the bytes per read by 2–4× and lower every floor accordingly.
- **Prior work.** Layer streaming from disk (AirLLM, FlexGen, ZeRO-Inference), tree speculation for offloaded models (SpecExec, Sequoia) and expert offloading (MoE-Infinity and others) are known techniques. The contribution here is the measured ranking on this hardware class, the decision rules that follow from it, and the observation that a plain Kubernetes memory limit already acts as an expert cache for mixture-of-experts models.

9. How to reproduce

```
git clone https://github.com/seneca-innovation-center/k8s-llm-offload-bench
cd k8s-llm-offload-bench
python3 scripts/reproduce_decision_table.py   # decision table (stdlib only)
python3 scripts/validate_output_identity.py   # token identity, all runs
pip install matplotlib markdown
python3 scripts/make_detailed_report.py       # 36 charts + DETAILED_REPORT
python3 scripts/build_docs.py                 # HTML and PDF of the reports
```

Re-running the measurements needs the cluster, the model weights and a few hours of machine time per model on this hardware, plus the model downloads; the steps are in `src/k8s/README.md`. The complete lab record, including every decision and every problem met, is `FULL_RECORD.md`.

Appendix A. Run inventory

Data file (under `results/`)	Content	Runs
`qwen2.5-3b/results.jsonl`	3B experiment 1: cyclic streaming, 18 configurations + controls	219
`qwen2.5-3b/results2.jsonl`	3B experiment 2: partial residency, kernel paging, speculative decoding, batching	195
`qwen2.5-3b/results3.jsonl`	3B single-pod streaming	63
`gpt-oss-20b/results.jsonl`	gpt-oss experiment 1	192
`gpt-oss-20b/results2.jsonl`	gpt-oss experiment 2	183

Data file (under `results/`)	Content	Runs
`qwen2.5-3b/results4.jsonl` … `results7.jsonl`	latency optimisation, rounds 1–4	33 + 72 + 72 + 75
`six-models/<model>.jsonl`	six-model test (Qwen2.5 1.5B/3B/7B/14B: 45 each; gpt-oss: 36; Qwen3-30B-A3B: 54)	270
`six-models/*_paging2.jsonl`	kernel-paging replication with the host cache cleared	45
`six-models/gptoss_paging_recheck.jsonl`	degenerate-prompt recheck	12
Total		**1,431**

Appendix B. Artifact log

Problems found during the work, and what was done about them. Each was caught before the numbers above were written.

Problem	How it was found	What was done	Effect on results
The page cache served "disk" reads in the first kernel-paging runs, twice	A 1.5 GiB pod ran at full speed	Deploy, then drop the cache, then measure	First two attempts discarded
Root-owned weight files blocked the cache drop (gpt-oss)	Peak memory 0.95 GB at every limit	Files made readable, 15 rows rerun	Rows replaced
Single-pod deadlock at 2048-token prompts	Thread dumps	Consecutive layer groups chained in-process	2 rows discarded
A repetitive prompt inflated speculative acceptance (3B)	100% draft acceptance	Non-repeating 750-token text	Artifact rows excluded
Direct I/O "faster than the link" (filesystem fallback to buffered reads)	11 GB/s on a 7.5 GB/s link; 48% of files in cache	Loader fixed to evict fallback pages	Round-1 direct-I/O rows invalid
Degenerate "…" output inflated gpt-oss kernel paging fourfold	Six-model test disagreed with the first figure	Replication and recheck (§6.3)	Corrected to 7.4–8.0×
Other workloads on the shared machine	Baselines up to 40% higher in some rounds	Same-round comparisons only	Flagged where it applies

Appendix C. Full tables

Every test, with its reasoning, full tables and chart, is in `DETAILED_REPORT.md` (36 tests). The raw data is in `results/`; `results/README.md` maps each file to the tests that use it.